

TripExpense SMS Verification

Final Architecture and Decision Document

****Status:**** Proposed / Architecture Baseline

****Date:**** 10 September 2026

****Project:**** TripExpense

****Primary development SMS gateway:**** Vendel, self-hosted

****Production SMS provider:**** Not selected yet

****Authentication:**** Better Auth

****Target market:**** India

1. Purpose

This document replaces the earlier MSG91-only implementation plan.

The goal is to add mandatory phone-number verification to TripExpense while keeping the SMS delivery layer independent from any single provider.

The immediate development path is:

TripExpense → Vendel Docker → Android phone / USB modem → SIM → SMS

A production provider such as MSG91 may be added later, but TripExpense must not depend directly on MSG91.

2. Decisions Made

Decision 1: Phone verification is mandatory

After email verification, a signed-in user who has not verified a phone number must complete phone verification before completing onboarding.

The resulting flow is:

Email signup

↓

Email verification

↓

Phone verification

↓

Username / profile / app lock

↓

Dashboard

There is no skip option for phone verification.

Decision 2: Better Auth remains responsible for OTP state

Better Auth's phone-number functionality is the authentication layer.

It should handle:

- OTP generation
- OTP storage
- OTP expiration
- OTP verification
- phone-number verification state

The SMS system is responsible only for delivering the generated OTP.

This separation is important because changing the SMS provider must not require rewriting authentication logic.

Decision 3: Do not couple TripExpense directly to MSG91

The previous plan called MSG91 directly from TripExpense.

That is no longer the architecture.

Instead:

Better Auth

↓

TripExpense SMS service

↓

SMS provider adapter

↓

Vendel / future provider

TripExpense should expose one internal operation conceptually equivalent to:

```
sendOtp(phoneNumber, otp)
```

The implementation behind it can change.

Decision 4: Vendel is the first SMS implementation

Vendel is being self-hosted with Docker.

Current local Vendel endpoint:

```
http://localhost:8090
```

Vendel exposes:

```
POST /api/sms/send
```

The TripExpense backend will authenticate to Vendel using an integration API key in:

X-API-Key

The Vendel API request is conceptually:

```
{  
  "recipients": ["+919876543210"],  
  "body": "Your TripExpense verification code is 123456"  
}
```

The Vendel key must remain server-side.

The Expo client must never receive it.

3. Vendel Responsibility

Vendel is the SMS gateway/device-management layer.

Vendel is responsible for:

- accepting SMS requests
- queueing/processing SMS
- communicating with the connected device
- sending through the Android phone or modem
- reporting SMS status

Vendel is NOT responsible for:

- TripExpense users
- Better Auth
- OTP generation
- OTP verification
- onboarding state
- TripExpense authorization

4. Hardware Model

Docker alone cannot send a cellular SMS.

The complete development setup requires a physical SMS-capable device.

Possible transport:

Vendel Docker

↓

Android phone + SIM

or:

Vendel Docker

↓

USB LTE/4G/5G modem + SIM

The Android-phone route is the preferred initial development path because it is easier to test.

5. DLT / Indian SMS Compliance Decision

DLT is an Indian telecom compliance concern and must be considered separately from Vendel.

We do NOT treat Vendel as a DLT bypass.

We also do NOT assume that a personal SIM is automatically suitable for production application OTP traffic.

Therefore:

Development

Use Vendel + personal/test device for controlled development and technical testing.

Production

Select a production SMS route after confirming the applicable Indian telecom requirements.

Possible provider:

MSG91

or another compliant SMS provider.

The production provider is deliberately left open.

6. Why We Are Not Making MSG91 the Foundation

The previous design assumed:

TripExpense → MSG91

This created a direct dependency on:

- MSG91 account setup
- MSG91 credentials
- MSG91 OTP template
- provider-specific API behavior
- Indian DLT/provider onboarding requirements

The new design avoids this coupling.

Instead:

TripExpense

↓

SMS abstraction

↓

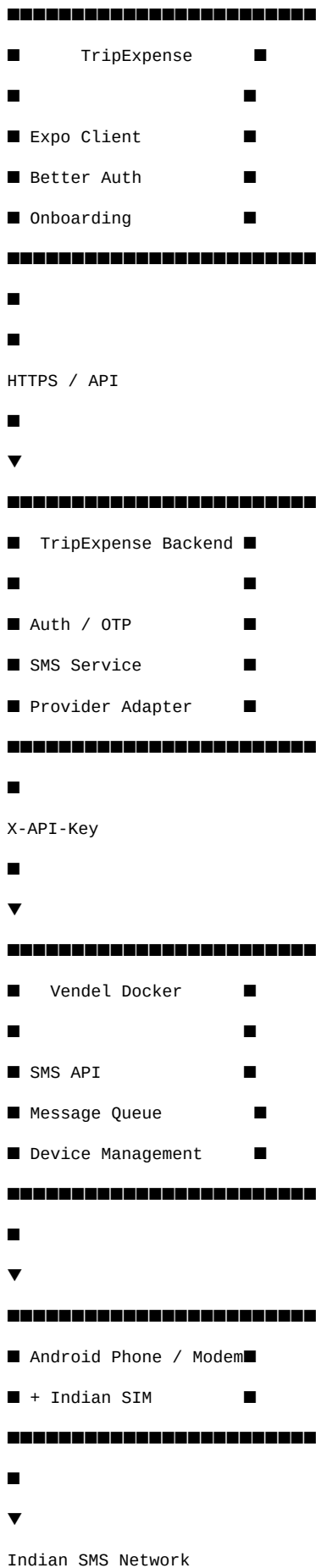
Vendel today

↓

Production provider later

This means the authentication system does not need to change if the SMS provider changes.

7. Final Architecture





User's Phone

8. Docker Networking

There are two possible development configurations.

Configuration A: TripExpense backend runs directly on Windows

Vendel:

`http://localhost:8090`

TripExpense backend can call:

`http://localhost:8090/api/sms/send`

Configuration B: TripExpense backend also runs in Docker

Do NOT use:

`http://localhost:8090`

from inside the TripExpense container.

Instead, both services should share a Docker network and TripExpense should call Vendel through its Compose service name.

Example:

`http://vendel:8090`

The exact service name will be determined from the final TripExpense Docker Compose configuration.

9. SMS API Contract

Vendel's send endpoint:

`POST /api/sms/send`

Authentication:

`X-API-Key: <Vendel integration API key>`

Request:

```
{
  "recipients": [
    "+919876543210"
  ],
  "body": "Your TripExpense verification code is 123456"
}
```

Optional fields supported by Vendel include:

```
{
  "recipients": ["+919876543210"],
  "body": "Your TripExpense verification code is 123456",
  "device_id": "",
  "group_ids": []
}
```

Expected accepted response:

```
{
  "batch_id": "...",
  "message_ids": ["..."],
  "recipients_count": 1,
  "status": "accepted"
}
```

Phone numbers should use E.164 format.

Example:

+919876543210

10. API Key Security

There are two different Vendel API-key concepts.

Integration API key

Used by TripExpense backend.

TripExpense backend

↓

X-API-Key

↓

Vendel

Device API key

Used by the Android/modem device to communicate with Vendel.

TripExpense must NOT use the device key.

Neither API key should ever be exposed to the Expo client.

11. OTP Flow

1. User signs up with email/password.
2. User verifies email.
3. Better Auth session is restored.
4. Auth phase checks `phoneNumberVerified`.
5. If false:
 route to phone verification.
6. User enters phone number.
7. Better Auth generates OTP.
8. Better Auth calls the configured `sendOTP` function.
9. TripExpense SMS service receives:
 recipient
 OTP code
10. SMS service calls Vendel.
11. Vendel sends SMS through the connected device.
12. User enters OTP.
13. Better Auth verifies OTP.

14. phoneNumberVerified becomes true.

15. User proceeds to normal onboarding.

16. User completes onboarding.

17. User reaches dashboard.

12. Authentication State Machine

The previous onboarding flow becomes:

SIGNED OUT



EMAIL SIGNUP



EMAIL VERIFICATION



SIGNED IN



phoneNumberVerified = false



PHONE VERIFICATION



phoneNumberVerified = true



ONBOARDING



DASHBOARD

Auth phase should contain:

needs-phone-verification

Expected phase logic:

User does not exist

→ signed-out/auth routes

User exists + email not verified

→ email verification

User exists + phone not verified

→ phone verification

User exists + phone verified + onboarding incomplete

→ existing onboarding

User exists + onboarding complete

→ dashboard

13. Database Changes

Better Auth's phone-number functionality requires user phone fields.

Expected fields:

phone_number

phone_number_verified

The exact schema must follow the Better Auth version already installed in TripExpense.

After schema changes:

```
npm run db:generate
```

```
npm run db:migrate
```

These commands must be verified against the actual TripExpense package scripts before execution.

14. Environment Configuration

TripExpense should have provider-neutral variables.

Recommended:

```
SMS_PROVIDER=vendel
```

```
VENDEL_URL=http://localhost:8090
```

VENDEL_API_KEY=<server-only-key>

If TripExpense backend and Vendel are both Docker services:

VENDEL_URL=http://vendel:8090

The final production provider can introduce its own variables later.

Example:

SMS_PROVIDER=msg91

without changing the authentication flow.

Do not commit real credentials.

15. Recommended Internal Structure

The exact paths must be checked against the current TripExpense repository before implementation.

Conceptually:

server/

■ ■ ■ ■ auth/

■ ■ ■ ■ auth-server

■

■ ■ ■ ■ sms/

■ ■ ■ ■ sms-service

■ ■ ■ ■ providers/

■ ■ ■ ■ vendel

■ ■ ■ ■ future-provider

The important rule is:

Better Auth

↓

SMS service

↓

Provider interface

↓

Vendel provider

Not:

Better Auth

↓

MSG91-specific code

16. SMS Message

The initial development message can be:

Your TripExpense verification code is 123456.

The final production message must follow the requirements of the selected production SMS route and applicable Indian telecom/DLT rules.

Therefore, the production template is intentionally NOT frozen yet.

17. Error Handling

The SMS service should distinguish between:

Provider unavailable

Vendel cannot be reached.

Result:

OTP send failed

The user should be allowed to retry.

Provider rejected request

Vendel rejects the request.

Result:

OTP send failed

Log the provider error server-side.

SMS accepted

Vendel returns:

status: accepted

TripExpense treats the send operation as accepted.

OTP expired

Better Auth handles OTP expiration.

The SMS provider does not determine OTP validity.

Wrong OTP

Better Auth handles verification failure.

The SMS provider is not involved.

18. Retry Policy

Do not blindly resend SMS every time the user presses the button.

The phone verification UI should have:

- resend cooldown
- clear error state
- OTP expiration
- controlled retries

The existing OTP cooldown mechanism in TripExpense should be reused where appropriate.

19. Development Testing

Stage 1: Test Vendel independently

Before changing TripExpense:

PowerShell

↓

Vendel API

↓

Android phone/modem

↓

test SMS

This proves the SMS infrastructure works.

Stage 2: Test TripExpense → Vendel

TripExpense backend

↓

Vendel API

↓

test SMS

This proves Docker/network/API authentication.

Stage 3: Test complete authentication

Signup

↓

Email verification

↓

Phone number

↓

OTP SMS

↓

OTP verification

↓

Onboarding

↓

Dashboard

20. Manual Acceptance Test

A new user should be able to:

1. Create an account.
2. Verify email.
3. Reach phone verification.

4. Enter an Indian phone number.
5. Receive an SMS.
6. Enter the OTP.
7. Successfully verify the phone.
8. Continue to username/profile onboarding.
9. Complete onboarding.
10. Reach the dashboard.
11. Sign out.
12. Sign back in.
13. Go directly to the dashboard if onboarding is already complete.

21. Security Requirements

Never expose:

`VENDEL_API_KEY`

to the Expo application.

Never put the Vendel API key in client-side code.

Never store the OTP in the client application beyond what the authentication flow requires.

OTP verification must remain server/auth-system controlled.

Use HTTPS for production backend communication.

Keep Vendel accessible only to the systems that need it.

22. Production Considerations

Vendel is the development/self-hosted SMS gateway in this architecture.

Before production launch, verify:

- Indian telecom requirements
- DLT requirements
- sender/header requirements
- message-template requirements

- SIM/operator terms
- expected SMS volume
- delivery reliability
- rate limits
- device availability
- monitoring
- failure handling

Do not assume the development phone/SIM setup is automatically a production-compliant commercial SMS system.

23. Why This Architecture Is Better

The original plan was:

TripExpense

↓

MSG91

The final architecture is:

TripExpense

↓

SMS abstraction

↓

Vendel today

↓

Production provider later

Benefits:

- No MSG91 dependency during development.
- No provider credentials in the client.
- Better Auth remains the authentication authority.
- Vendel can be tested locally.
- Production provider can change later.
- DLT/provider decisions remain outside the authentication core.
- SMS infrastructure can evolve independently.

24. Current Project Status

Completed / Decided

- Phone verification is mandatory.
- Better Auth remains responsible for OTP generation and verification.
- MSG91 is no longer the foundational SMS implementation.
- Vendel has been selected as the development SMS gateway.
- Vendel is running through Docker.
- Vendel API is exposed on port 8090.
- Vendel's SMS send endpoint has been identified.
- Integration API key authentication has been identified.
- Android phone/modem is required for actual cellular SMS.
- Provider abstraction is required.
- DLT is treated as a production/compliance concern, not as a Vendel feature.

Not yet implemented

- TripExpense code changes.
- Better Auth phone plugin integration.
- TripExpense database migration.
- SMS provider abstraction.
- Vendel provider adapter.
- TripExpense → Vendel Docker networking.
- Android/modem registration and final SMS test.
- Production SMS provider selection.

25. Implementation Order

Do the work in this order.

STEP 1

Verify Vendel independently

↓

STEP 2

Connect Android phone/modem to Vendel

↓

STEP 3

Send a manual test SMS through Vendel

↓

STEP 4

Inspect TripExpense Docker/backend setup

↓

STEP 5

Connect TripExpense backend to Vendel

↓

STEP 6

Create SMS provider abstraction

↓

STEP 7

Integrate Better Auth phone verification

↓

STEP 8

Add database migration

↓

STEP 9

Add phone-verification onboarding route

↓

STEP 10

Update auth-phase routing

↓

STEP 11

Run complete authentication test

↓

STEP 12

Document production SMS/DLT decision

26. Final Target

The final development system should behave like this:

USER

■

▼

TripExpense App



TripExpense API



Better Auth



generates OTP



SMS Service



Vendel Adapter



X-API-Key



Vendel Docker



Android Phone / Modem



SIM



SMS



USER

The key architectural principle is:

> **TripExpense owns authentication. Vendel owns SMS delivery. The SMS provider is replaceable.**

This is the baseline to use before implementation begins.